

Đề bài: Xây dựng Inventory Service trong hệ thống Microservices

Mục tiêu:

- Hiểu và áp dụng kiến thức về Spring Boot để xây dựng một microservice quản lý kho hàng.
 - Sử dụng **Spring Data JPA** để lưu trữ thông tin kho hàng trong cơ sở dữ liệu quan hệ (MySQL hoặc PostgreSQL).
 - Tích hợp với **Eureka Discovery Server** để đăng ký và quản lý service.
 - Cung cấp API để kiểm tra tình trạng tồn kho của các sản phẩm.
 - Thực hành viết unit test và integration test.
 - Tích hợp **Actuator**, **Prometheus**, và **Zipkin** để giám sát và theo dõi hiệu suất ứng dụng.
-

Yêu cầu chi tiết:

1. Tạo ứng dụng Spring Boot

- Sử dụng **Spring Initializr** để khởi tạo dự án với các dependency:
 - Spring Web
 - Spring Data JPA
 - Spring Cloud Eureka Client
 - Lombok
 - Spring Boot Actuator
 - Micrometer Tracing (Zipkin)
 - Micrometer Registry Prometheus
- Cấu hình ứng dụng để kết nối với cơ sở dữ liệu (MySQL hoặc PostgreSQL).

2. Xây dựng các REST API

- **Kiểm tra tình trạng tồn kho:**
 - API: GET /api/inventory
 - Query parameters: skuCode (danh sách các mã sản phẩm cần kiểm tra).
 - Response:

```
[
  {
    "skuCode": "iphone_13",
    "isInStock": true
  },
  {
    "skuCode": "iphone_13_red",
```

```
"isInStock": false
}
```

3. Sử dụng cơ sở dữ liệu quan hệ

- Tạo model Inventory với các trường:
 - id (Long)
 - skuCode (String)
 - quantity (Integer)
- Sử dụng **Spring Data JPA** để thao tác với cơ sở dữ liệu.

4. Tích hợp với Eureka Discovery Server

- Cấu hình application.properties để đăng ký service với Eureka Server:

```
eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
spring.application.name=inventory-service
```
- Đảm bảo service có thể được discovery bởi các service khác trong hệ thống.

5. Tích hợp Actuator và Prometheus

- Cấu hình Actuator để expose các endpoint:
 - Health check: /actuator/health
 - Metrics: /actuator/prometheus
- Sử dụng **Micrometer** để tích hợp với Prometheus và Zipkin.

6. Viết unit test và integration test

- Viết test cho các API:
 - Test kiểm tra tình trạng tồn kho thành công.
 - Test kiểm tra tình trạng tồn kho với sản phẩm không tồn tại.
- Đảm bảo test coverage tối thiểu 80%.

7. Triển khai ứng dụng với Docker

- Tạo Dockerfile để đóng gói ứng dụng.
- Sử dụng Docker Compose để chạy ứng dụng cùng với cơ sở dữ liệu và Eureka Server.

Gợi ý:

1. Công cụ và công nghệ sử dụng:

- Spring Boot
- Spring Data JPA
- Spring Cloud Eureka Client
- Lombok
- Actuator, Prometheus, Zipkin
- Docker

2. Tài liệu tham khảo:

- Tài liệu chính thức của Spring Boot: <https://spring.io/projects/spring-boot>
- Hướng dẫn sử dụng Spring Data JPA: <https://spring.io/guides/gs/accessing-data-jpa/>
- Hướng dẫn sử dụng Eureka: <https://spring.io/guides/gs/service-registration-and-discovery/>

3. Cấu trúc dự án tham khảo:

```
Copy
inventory-service/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com.hdbank.inventoryservice/
│   │   │   │   ├── controller/
│   │   │   │   ├── model/
│   │   │   │   ├── repository/
│   │   │   │   ├── service/
│   │   │   │   ├── dto/
│   │   │   │   ├── util/
│   │   │   │   └── InventoryServiceApplication.java
│   │   └── resources/
│   │       ├── application.properties
│   │       └── banner.txt
│   └── test/
│       ├── java/
│       │   ├── com.hdbank.inventoryservice/
│       │   └── InventoryServiceApplicationTests.java
└── pom.xml
```

Phần mở rộng (nếu có thời gian):

- Thêm tính năng **Logging** và **Tracing** sử dụng **ELK Stack** hoặc **Jaeger**.
- Triển khai ứng dụng lên **Kubernetes**.
- Thêm tính năng **Authentication** và **Authorization** sử dụng **Spring Security**.